v1.0.0
20260302

Programmazione Concorrente e Distribuita

Ingegneria e Scienze Informatiche - UNIBO
a.a 2025/2026
Prof. Alessandro Ricci

[Lab Notes]
THREAD LIVENESS

BACK TO PERFORMANCE: POOR CONCURRENCY PROBLEM

- The misuse of atomic blocks can lead to performance problems.
  - sequentialisations => strong impact on speed-up
    - poor performances
- => Careful choice of what parts must be designed and implemented as critical sections
  - minimize the time in which a lock is retained
    - but don’t trade performance with safety

AVOIDING LIVENESS HAZARD

rischi

- Tension between *safety* and *liveness*

gli strumenti che usiamo per rendere il programma sicuro (Safety) sono gli stessi che possono bloccarlo completamente (Liveness).

  - using locking to ensure thread safety
    - ...but indiscriminate use of locking can cause "lock-ordering" deadlocks

Si verifica quando due thread cercano di acquisire gli stessi due lock, ma lo fanno in ordine diverso (dining philosophers).

  - using thread pools and semaphores ~~to bound~~ per limitare resource consumption
    - ...but ~~failure to understand activities being bounded~~ una comprensione errata delle attività soggette a limitazione can cause resource deadlocks

Se non capisci bene come le attività interagiscono, potresti creare uno stallo.

Esempio: Thread Pool Deadlock
Immagina un pool con un solo thread disponibile:

Il Thread A (nel pool) inizia un task.

Per finire, il task del Thread A deve aspettare il risultato di un secondo task (B).

Il task B viene messo in coda nel pool, aspettando che un thread si liberi.

Ma l'unico thread è occupato da A, che sta aspettando B.

Risultato: Il sistema è paralizzato perché la risorsa necessaria per sbloccare la situazione è occupata da chi sta aspettando lo sblocco.

DEADLOCKS

- A situation wherein *two or more competing actions are waiting for the other to finish, and thus neither ever does*
- Coffman *necessary conditions* for a deadlock to occur (1971)
  - **mutual exclusion condition**
    - a resource that cannot be used by more than one process at a time
  - **hold and wait condition**
    - processes already holding resources may request new resources
  - **no preemption condition**
    - no resource can be removed from a process holding it
    - resources can be released only by the explicit action of the process
  - **circular wait condition**
    - two or more processes form a circular chain where each process waits for a resource that the next process in the chain

DEADLOCKS WITH LOCKS

- It happens when
  - multiple threads wait forever due to cyclic locking dependency
  - simplest case
    - when thread A holds lock L and tries to acquire lock M, but at the same time thread B holds M and tries to acquire L, both thread will wait for ever
    - deadly embrace

DEADLOCKS DETECTION & RECOVERY

- Deadlocks **detection** and **recovery**
  - adopted in databases
    - databases are designed to detect and recover from deadlocks
    - transactions typically acquire many locks, until they commit
    - not so uncommon for two transactions to deadlock
  - identifying the set of transactions that are deadlocked by analyzing *is-waiting* dependency graph
    - looking for cycles
    - if a cycle is found, a victim is selected and the transaction aborted
- No automated deadlock detection / recovery mechanism in JVM
  - if threads deadlock, *that's all folks*
    - we can just shutdown the application
  - "post-mortem" diagnosis support

DEADLOCK DIAGNOSING

- *Thread dump* support provided by the JVM

snapshots of the current state of all Java threads in a Java Virtual Machine (JVM) at a specific point in time.

  - triggered by
    - sending the JVM process a SIGQUIT signal on UNIX (kill -3)
    - pressing CTRL-\ on UNIX
    - pressing CTRL-Break on Windows
- Thread dump content
  - stack trace for each running thread
  - locking information
    - which locks are held by each thread, in which stack frame they were acquired, and which lock a blocked thread is waiting for

LOCK-ORDER DEADLOCKS

- Example: Transfer money between bank accounts

```
public class Test1a {

  private static final int NUM_THREADS = 20;
  private static final int NUM_ACCOUNTS = 5;
  private static final int NUM_ITERATIONS = 10000000;
  private static final Random gen = new Random();
  private static final Account[] accounts = new Account[NUM_ACCOUNTS];

  public static void transferMoney(Account from,  Account to, int amount)
                                        throws InsufficientBalanceException {
    synchronized (from) {
      synchronized (to) {
        if (from.getBalance() < amount)
          throw new InsufficientBalanceException();
        from.debit(amount);
        to.credit(amount);
      }
    }
  }

  ...
}
```

```java
public class Test1a {

  private static final int NUM_THREADS = 20;
  private static final int NUM_ACCOUNTS = 5;
  private static final int NUM_ITERATIONS = 10000000;
  private static final Random gen = new Random();
  private static final Account[] accounts = new Account[NUM_ACCOUNTS];

  public static void transferMoney(Account from,  Account to, int amount)
            throws InsufficientBalanceException {...}

  static class TransferThread extends Thread {
    public void run() {
      for (int i = 0; i < NUM_ITERATIONS; i++){
        int fromAcc = gen.nextInt(NUM_ACCOUNTS);
        int toAcc = gen.nextInt(NUM_ACCOUNTS);
        int amount = gen.nextInt(10);
        try {
          transferMoney(accounts[fromAcc],accounts[toAcc],amount);
        } catch (InsufficientBalanceException ex){}
      }
    }
  }

  public static void main(String[] args) {
    for (int i = 0; i < accounts.length; i++){
      accounts[i] = new Account(1000);
    }
    for (int i = 0; i < NUM_THREADS; i++){
      new TransferThread().start();
    }
  }
}
```

ORDERING LOCKS

- Deadlock ~~came~~ si è verificato because two threads attempted to acquire the locks *in a different order*
  - if they asked for the locks in the same order, there would be no cyclic locking dependency and therefore no deadlock
- A program will be free of lock-ordering deadlocks if all threads acquire the locks they need in a fixed global order
  - verifying consistent lock ordering requires a global analysis of your program's locking behavior

```java
public class AccountManager {

  private final Account[] accounts;

  public AccountManager(int nAccounts, int amount){
    accounts = new Account[nAccounts];
    for (int i = 0; i < accounts.length; i++){
      accounts[i] = new Account(amount);
    }
  }

  public void transferMoney(int from, int to, int amount)
                                    throws InsufficientBalanceException {

    int first = from;
    int last = to;

    if (first > last){
      last = first;
      first = to;
    }

    synchronized (accounts[first]) {
      synchronized (accounts[last]) {
        if (accounts[from].getBalance() < amount)
          throw new InsufficientBalanceException();
        accounts[from].debit(amount);
        accounts[to].credit(amount);
      }
    }
  }
}
```

```
public class Test1b {

  private static final int NUM_THREADS = 20;
  private static final int NUM_ACCOUNTS = 5;
  private static final int NUM_ITERATIONS = 10000000;
  private static final Random gen = new Random();

  static class TransferThread extends Thread {
    AccountManager man;
    TransferThread(AccountManager man){
      this.man = man;
    }
    public void run() {
      for (int i = 0; i < NUM_ITERATIONS; i++){
        int fromAcc = gen.nextInt(NUM_ACCOUNTS);
        int toAcc = gen.nextInt(NUM_ACCOUNTS);
        int amount = gen.nextInt(10);
        try {
          man.transferMoney(fromAcc,toAcc,amount);
        } catch (InsufficientBalanceException ex){
        }
      }
    }
  }
  public static void main(String[] args) {
    AccountManager man = new AccountManager(NUM_ACCOUNTS,1000);
    for (int i = 0; i < NUM_THREADS; i++){
      new TransferThread(man).start();
    }
  }
}
```

DEADLOCKS BETWEEN COOPERATING OBJECTS

- More subtle deadlocks can happen in cooperating objects, in which no methods explicitly acquire two locks, but where this happens indirectly
- A common example: *Observer pattern*
  - observers observing observed object
  - different control flows executing methods for observers and observed
- The more general problem
  - event-oriented pattern implementation in OO languages
    - coupling controls among sources and observers of events
  - MVC case study

```java
interface IObserved {
  int getState();
  void register(IObserver obj);
}

class MyEntityA implements IObserved {

  private List<IObserver> obsList;
  private int state;

  public MyEntityA(){
    obsList = new ArrayList<IObserver>();
  }

  public void register(IObserver obs) {
    obsList.add(obs);
  }

  public synchronized int getState() {
    return state;
  }

  public synchronized void changeState1() {
    state++;
    for (IObserver o: obsList){
      o.notifyStateChanged(this);
    }
  }

  public synchronized void changeState2() {
    state--;
    for (IObserver o: obsList){
      o.notifyStateChanged(this);
    }
  }
}
```

```java
interface IObserver {
  void notifyStateChanged(IObserved obs);
}

class MyEntityB implements IObserver {

  List<IObserved> obsList;

  public MyEntityB(){
    obsList = new ArrayList<IObserved>();
  }

  public synchronized void observe(IObserved obj){
    obsList.add(obj);
    obj.register(this);
  }

  public synchronized
             void notifyStateChanged(IObserved obs) {
    synchronized(System.out){
      System.out.println(
            "state changed: "+obs.getState());
    }
  }

  public synchronized int getOverallState() {
    int sum = 0;
    for (IObserved o: obsList){
      sum += o.getState();
    }
    return sum;
  }
}
```

```java
class MyThreadA extends Thread {
  MyEntityA obj;

  public MyThreadA(MyEntityA obj){
    this.obj = obj;
  }

  public void run(){
    while (true){
      obj.changeState1();
      obj.changeState2();
    }
  }
}

class MyThreadB extends Thread {
  MyEntityB obj;

  public MyThreadB(MyEntityB obj){
    this.obj = obj;
  }

  public void run(){
    while (true){
      log("overall state: "+
                obj.getOverallState());
    }
  }

  private void log(String msg){
    synchronized(System.out){
      System.out.println("["+this+"] "+msg);
    }
  }
}
```

```java
public class Test2 {
  public static void main(String[] args) {

    MyEntityA objA = new MyEntityA();
    MyEntityB objB = new MyEntityB();
    objB.observe(objA);

    new MyThreadA(objA).start();
    new MyThreadB(objB).start();

  }
}
```

AVOIDING DEADLOCKS

- A program that never acquires more than one lock a time cannot have lock-ordering deadlock
- if we must acquire multiple locks, lock ordering must be part of the design
  - minimizing the number of potential locking interaction
  - document a lock-ordering protocol for locks that may be acquired together
- Alternative technique: *timed locks*
  - detecting and recovering from deadlocks using `tryLock` feature of `Lock` classes instead of intrinsic lock

OTHER LIVENESS HAZARD

- **Starvation**
  - typically manifested when using *priorities*
  - basic thread support for priorities in Java thread is “deprecated”
    - platform dependent
    - liveness problems
- **Poor responsiveness**
  - e.g. executing long-term tasks in GUI thread
  - can also be caused by *poor lock management*
    - if a thread holds a lock for long time - for instance while iterating on a large collection and performing substantial work on each element - other threads that need to access that collection may have to wait long time
- **Livelock**
  - when threads cannot make progress because they keep retrying an operation that will always fail
  - solution: introducing some *randomness* into the retry mechanism
    - breaking the synchronization that causes the live-lock

BIBLIOGRAPHY

- *“Java Concurrency in Practice”*, Brian Goetz, Addison Wesley